

Titulación: Grado en Matemáticas y Computación
Curso: 2025-2026. Convocatoria Ordinaria de Junio
Asignatura: Bases de Datos Avanzadas – Laboratorio

Practica 4: Replicación e Implementación de una Base de Datos Distribuida.

ALUMNO 1:

Nombre y Apellidos: José Antonio López Gonnzález

DNI: 03209889D

ALUMNO 2:

Nombre y Apellidos: Jorge López Duque

DNI: 70943373B

Fecha: 27/05/2026

Profesor Responsable: Javier Arroyo Gallardo (Ahora bien escrito)

Mediante la entrega de este fichero los alumnos aseguran que cumplen con la normativa de autoría de trabajos de la Universidad de Alcalá, y declaran éste como un trabajo original y propio.

En caso de ser detectada copia, se puntuará **TODA** la asignatura como Suspenso – Cero.

Plazos

Tarea en laboratorio: Semana 20 de abril, Semana 27 de abril, Semana 4 de mayo y semana 11 de mayo.

Entrega de práctica: **Día 17 de mayo.** Aula Virtual

Documento a entregar: Este mismo fichero con la implementación de la replicación y la base de datos distribuida, las pruebas realizadas de su funcionamiento; y los ficheros de configuración del maestro y del esclavo utilizados en replicación; y de la configuración de los servidores de la base de datos distribuida (obligatorio). También los ficheros de log generados en la realización de la práctica de los postgres involucrados (obligatorio). Fichero para subir DNI's delos Alumnos_Pecl4.zip

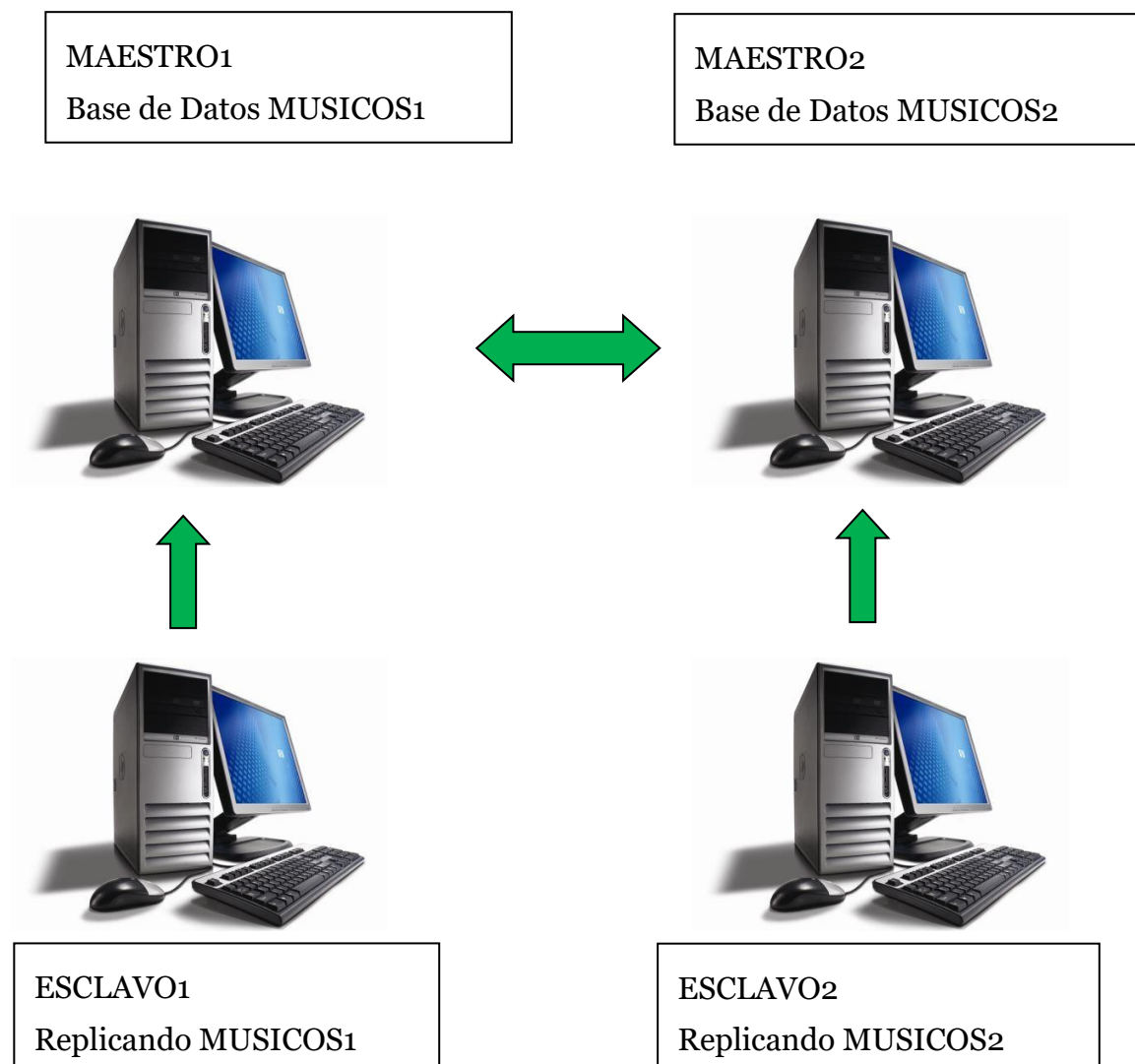
Los profesores del laboratorio podrán solicitar una presentación/defensa de la práctica para ver que todo funciona correctamente.

Introducción

El contenido de esta práctica versa sobre la Replicación de Bases de Datos con PostgreSQL e introducción a las bases de datos distribuidas. Concretamente se va a utilizar los servicios de replicación de bases de datos que tiene PostgreSQL. Para ello se utilizará PostgreSQL 18.x con soporte para replicación. **Se prohíbe el uso de cualquier otro programa externo a PostgreSQL para realizar la replicación, como puede ser Slony.**

También se va a diseñar e implementar una pequeña base de datos distribuida. Una base de datos distribuida es una base de datos lógica compuesta por varios nodos (equipos) situados en un lugar determinado, cuyos datos almacenados son diferentes; pero que todos ellos forman una base de datos lógica. Generalmente, los datos se reparten entre los nodos dependiendo de donde se utilizan más frecuentemente.

El escenario que se pretende realizar se muestra en el siguiente esquema:



Se van a necesitar 4 postgres: 2 maestros y 2 esclavos. Cada maestro puede ser un ordenador de cada miembro del grupo con una base de datos de MUSICOS en concreto (MUSICOS1 y MUSICOS2). Dentro de cada maestro se puede instalar una máquina virtual, que se corresponderá con el esclavo que se encarga de replicar la base de datos que tiene cada maestro, es decir, hace una copia o backup continuo de la base de datos MUSICOS1 o de la base de datos MUSICOS2. También es posible no usar otra máquina virtual para el esclavo y levantar otra estancia de postgres para guardar la réplica en un clúster diferente. Hay más de una solución.

Se debe de entregar el cuestionario que posea los siguientes puntos para poder ser reproducido el trabajo por cualquier persona que pueda leerlo.

Cuestión o. Configurar el fichero de Error Reporting and Logging de PostgreSQL para que aparezcan recogidas las sentencias SQL DDL (Lenguaje de Definición de Datos) + DML (Lenguaje de Manipulación de Datos) generadas en dicho fichero. No se pide activar todas las sentencias. No activar la duración de la consulta. También se debe de configurar el log para que en el comienzo de la línea de registro de la información del log (“line prefix”) aparezcan los DNI’s de los miembros del equipo, el nombre del host con su puerto y la **fecha y hora** de la operación.

Se necesita la siguiente configuracion:

logging_collector = on

-Registrar DDL + DML de modificación:

log_statement = 'mod'

-No registrar duración:

log_duration = off

log_min_duration_statement = -1

-DNI + host + puerto + fecha/hora:

log_line_prefix = '70943373B_03209889D | %r | %t | '

Basicamente no se han hecho cambios de la configuracion de la practica anterior.

Acceso a Bases de Datos Remotas usando el método del Apéndice F38. Postgres_fdw

Cuestión 1. Configuración de cada uno de los nodos maestros de la base de datos de MUSICOS1 y MUSICOS2 por medio del **módulo postgres_fdw** que se puedan recibir y realizar consultas sobre las tablas de la base de datos que no tienen implementadas localmente.

En postgresql.conf:

-Permitir escuchar conexiones externas:

```
listen_addresses = '*'
```

```
port = 5432
```

En pg_hba.conf: Para autorizar que el PC del compañero entre al otro PostgreSQL:

```
host postgres all 111.111.111.111/32 md5
```

(Cambiar ips en cada cual)

Para configurar la conexin remota:

```
CREATE SERVER maestro2_fdw
FOREIGN DATA WRAPPER postgres_fdw
OPTIONS (
  host '192.168.50.170',
  dbname 'postgres',
  port '5432'
);
```

CREATE USER MAPPING FOR postgres

SERVER maestro2_fdw

OPTIONS (

```
  user 'postgres',
  password '1234'
```

);

Para pillar las tablas de manera remota:

```
IMPORT FOREIGN SCHEMA public
```

```
EXCEPT (canciones, discos, grupo)
```

```
FROM SERVER maestro2_fdw
```

```
INTO public;
```

Para comprobar datos de tablas foraneas:

```
SELECT * FROM conciertos;
```

Cuestión 2. Insertar datos diferentes en cada una de las bases de datos del MAESTRO1 y del MAESTRO2. Realizar una consulta sobre el MAESTRO1 que

permita obtener el número total de canciones de cada grupos junto con el número de conciertos realizados por cada grupo. Se debe de usar toda la base de datos distribuida (MAESTRO1 + MAESTRO2) en una sola consulta SQL a realizar en el MAESTRO1. Explicar cómo se resuelve la consulta y su plan de ejecución.

Consulta

```
SELECT
  g.Codigo_grupo,
  g.Nombre,
  COUNT(DISTINCT c.Codigo_cancion) AS total_canciones,
  COUNT(DISTINCT gc.Codigo_concierto) AS total_conciertos
FROM Grupo g
LEFT JOIN Discos d
  ON d.Codigo_grupo_grupo = g.Codigo_grupo
LEFT JOIN Canciones c
  ON c.Codigo_disco_discos = d.Codigo_disco
```

Plan de Ejecución

```
GroupAggregate (cost=1019.87..1062.62 rows=510 width=52)
" Output: g.codigo_grupo, g.nombre, count(DISTINCT c.codigo_cancion), count(DISTINCT
gc.codigo_concierto_conciertos)"
  Group Key: g.codigo_grupo
    -> Sort (cost=1019.87..1029.28 rows=3765 width=44)
      " Output: g.codigo_grupo, g.nombre, c.codigo_cancion, gc.codigo_concierto_conciertos"
      " Sort Key: g.codigo_grupo, c.codigo_cancion"
        -> Hash Right Join (cost=175.81..796.26 rows=3765 width=44)
          " Output: g.codigo_grupo, g.nombre, c.codigo_cancion, gc.codigo_concierto_conciertos"
          Hash Cond: (gc.codigo_grupo_grupo = g.codigo_grupo)
            -> Foreign Scan on public.grupos_tocan_conciertos gc (cost=100.00..673.20 rows=2560
width=8)
              " Output: gc.codigo_grupo_grupo, gc.codigo_concierto_conciertos"
              " Remote SQL: SELECT codigo_grupo_grupo, codigo_concierto_conciertos FROM
public.grupos_tocan_conciertos"
```

Explicaciones y Comentarios

La consulta se ejecuta en el nodo MAESTRO1 combinando datos locales y remotos. Las tablas Grupo, Discos y Canciones se procesan localmente mediante escaneos secuenciales y operaciones de hash join.

La tabla Grupos_Tocan_Conciertos se accede de forma remota mediante postgres_fdw, lo que se refleja en la operación Foreign Scan del plan de ejecución.

El optimizador de PostgreSQL construye una estrategia basada en hash joins para mejorar el rendimiento de las combinaciones entre tablas.

Posteriormente se realiza una ordenación de los datos y una agregación mediante GroupAggregate para calcular el número de canciones y conciertos por grupo.

El uso de COUNT(DISTINCT) evita duplicidades generadas por la multiplicación de filas en las uniones entre relaciones 1:N (discos-canciones y grupos-conciertos).

El plan de ejecución demuestra claramente la distribución del sistema, ya que la tabla grupos_tocan_conciertos se obtiene mediante un Foreign Scan desde MAESTRO2 utilizando postgres_fdw, mientras que el resto de operaciones se realizan localmente en MAESTRO1.

Replicación Física usando método capítulo 26 de replicación.

Cuestión 3. Configuración completa de los equipos para estar en modo de replicación. Configuración del nodo maestro y del nodo esclavo. Tipos de nodos maestros, diferencias en el modo de funcionamiento y tipo elegido. Tipos de nodos esclavos, diferencias en el modo de funcionamiento y tipo elegido, etc.

Desde cmd como admin al /bin del postgres y comando cd /d "C:\Program Files\PostgreSQL\18\bin"

Ejecutar initdb -D "C:\postgres_esclavo"

En el hba del maestro hemos metido el replicator: host replication replicator 192.168.50.140/32 md5

En el config del maestro hemos activado la replicacion.

En el datagrip desde el maestro hemos creado el usuario replicator con su contraseña
hemos reiniciado postgres: CREATE USER replicator WITH REPLICATION PASSWORD '1234';

Vaciamos por comando al esclavo.

Hacemos una copia por cmd del maestro en el esclavo con el comando:
pg_basebackup -h 192.168.50.170 -D C:\postgres_esclavo -U replicator -P -R

Cambiamos el puerto del esclavo al 5433 y lo iniciamos por cmd: pg_ctl -D C:\postgres_esclavo start

Creamos el datasource esclavo y hacemos las pruebas:

-En maestro:
SELECT inet_server_port();
--Deberia dar false:
SELECT pg_is_in_recovery();
--Ver si esclavo esta conectado:
SELECT * FROM pg_stat_replication;

-En esclavo:
SELECT inet_server_port();
--Deberia dar true:
SELECT pg_is_in_recovery();
--Comprobacion de esclavo solo lectura:
SELECT * FROM Grupo WHERE codigo_grupo = 4;

El nodo maestro es el servidor principal en el que se realizan todas las operaciones de escritura de la base de datos y donde se generan los registros WAL, que son los que permiten la replicación. Este nodo controla la consistencia de los datos y es el único que admite modificaciones en una replicación física. Por otro lado, el nodo esclavo es una copia del maestro que no permite escrituras en condiciones normales, ya que su función es recibir los registros WAL generados por el maestro y aplicarlos en el mismo orden para mantener una copia idéntica de la base de datos. Su rol principal es mantener la sincronización con el nodo maestro.

El tipo de nodo maestro más habitual, en PSQL, es el maestro único, donde solo un servidor gestiona todas las escrituras, que simplifica la arquitectura y evita conflictos de datos, mientras que el modelo multi-maestro (varios nodos pueden escribir simultáneamente) no es típico en PSQL clásico y requiere mecanismos complejos de resolución de conflictos.

De nodos esclavos, existe el hot standby, que permite realizar consultas de lectura mientras se está replicando activamente; el warm standby, que recibe datos pero no se utiliza activamente para consultas durante la sincronización; y el cold standby, que permanece apagado o inactivo hasta que se necesita en caso de fallo del maestro.

En términos generales, la replicación física se basa en la copia a nivel de bloques del sistema de almacenamiento de PostgreSQL, lo que significa que no se replica el SQL como tal, sino los cambios internos registrados en los WAL, asegurando así que el nodo esclavo sea una réplica exacta del

Cuestión 4. Comentar los tipos de operaciones que se pueden realizar en cada tipo de nodo. Además, provocar situaciones de caída de los nodos y observar los mensajes obtenidos por cada nodo. Realizar las acciones correctoras necesarias para volver el sistema a un estado normal de operaciones. Comentar lo observado y realizado en esta cuestión.

1) Tipos de operaciones en cada nodo:

-Nodo Maestro: El nodo maestro es el servidor principal de PostgreSQL.

Operaciones permitidas: SELECT, INSERT, UPDATE, DELETE, CREATE TABLE, ALTER TABLE, DROP TABLE, escritura de WAL (Write Ahead Log) y envío de cambios al esclavo

-Nodo Esclavo: El nodo esclavo es una réplica física del maestro.

Operaciones permitidas: SELECT

Operaciones NO permitidas: INSERT, UPDATE, DELETE, CREATE TABLE

2) Situación de caída del nodo esclavo:

Parar PostgreSQL del esclavo.

El maestro sigue funcionando normalmente, pero pg_stat_replication en el maestro ya no muestra streaming activo.

Comprobación en maestro: SELECT * FROM pg_stat_replication;

Recuperación, arrancamos otra vez el esclavo.

Luego comprobar: SELECT pg_is_in_recovery() (true)

Y en maestro: SELECT * FROM pg_stat_replication. (state = streaming)

3) Situación de caída del nodo maestro:

Parar PostgreSQL del maestro.

En el maestro no responde conexiones.

En el esclavo sigue accesible en modo lectura.

Arrancar el maestro otra vez.

El maestro vuelve a generar WAL y el esclavo vuelve a sincronizar.

Comprobación:

-En maestro: INSERT INTO Grupo VALUES (3000, 'Tell the Avivs', 'Pop', 'IS', 'Israeleslegitimo.com');

-En esclavo: SELECT * FROM Grupo WHERE codigo_grupo = 3000; Si aparece el sistema se ha restaurado.

Durante la práctica se ha observado que el nodo maestro es el único que admite operaciones de escritura, mientras que el nodo esclavo permanece en modo solo lectura y replica continuamente los cambios recibidos mediante WAL.

Cuando el nodo esclavo cae, el maestro continúa funcionando de forma normal, aunque deja de existir sincronización temporal hasta que el esclavo vuelve a conectarse.

Cuando el nodo maestro cae, el esclavo sigue permitiendo operaciones de lectura, pero no puede recibir nuevos cambios, por lo que la replicación queda detenida.

Tras reiniciar el nodo afectado, el sistema recupera el estado normal y la replicación streaming vuelve a establecerse correctamente.

Replicación lógica usando el procedimiento del capítulo 29

Cuestión 5. Configuración completa de los equipos para estar en modo de replicación lógica. Configuración de ambos tipos de nodos, funcionamiento de ambos tipos de nodos, etc.

La replicación lógica en PostgreSQL permite replicar cambios a nivel lógico realizadas sobre tablas concretas.

-Proceso:

Cambiamos el wal level del maestro a logical para conseguir replicacion fisca + logica.

Abrimos la carpeta del bin en el cmd.

Creamos el esclavo CON EL USUARIO con initdb -D C:\esclavo_logico -U postgres.

Inicializamos el servidor: pg_ctl -D C:\esclavo_logico -o "-p 5434" start

Cambiamos el puerto y la escucha del esclavo logico (Port = 5434, wal_level = logical)

Reiniciamos el servidor del esclavo

En el hba del master cambiamos para incluir al esclavo: host all miguelito 192.168.50.170/32 md5

Reiniciamos el postgres

Creamos a miguelito y le damos permisos en el sql

Creamos el datasource

Y proseguimos con publicacion y subscripcion:

-En maestro (publisher):

--Crear publicacion:

```
CREATE PUBLICATION pub_grupos
```

```
FOR ALL TABLES;
```

--Comprobaciones:

```
SELECT * FROM pg_publication;
```

--Sentencia de insert:

```
INSERT INTO Grupo VALUES (5,'Neta ´n the Yahus', 'Ska religioso', 'Israhell', 'BigYahu.is');
```

-En esclavo (subscriber):

Primero creamos todas las tablas que tien la base de datos de postgre, puesto que la subscripcion no las crea.

Y luego:

--Crear subscripcion:

```
CREATE SUBSCRIPTION sub_grupos
```

```
CONNECTION 'host=192.168.50.140 port=5432 dbname=postgres user=postgres
```

```
password=Jose83471'
```

```
PUBLICATION pub_grupos;
```

--Comprobaciones:

```
SELECT * FROM pg_subscription;
```

--Comprobar la nueva insercion:

```
SELECT * FROM Grupo WHERE Codigo_grupo = 5;
```

La replicación lógica proporciona una mayor flexibilidad que la física, ya que permite seleccionar únicamente determinadas tablas para replicar.

Además, el subscriber no necesita ser una copia física exacta del publicador, permitiendo configuraciones más personalizadas.

Durante la práctica hemos visto que los cambios realizados sobre las tablas publicadas en el publicador se replican automáticamente en el subscriber mediante publicaciones y suscripciones.

La replicación lógica en PostgreSQL replica cambios a nivel de tablas mediante publicaciones y suscripciones, proporcionando mayor flexibilidad que la replicación física.

Cuestión 6. Comentar los tipos de operaciones que se pueden realizar en cada tipo de nodo. Además, provocar situaciones de caída de los nodos y observar los mensajes obtenidos por cada nodo. Realizar las acciones correctoras necesarias para volver el sistema a un estado normal de operaciones. Comentar lo observado y realizado en esta cuestión.

1)Tipos de operaciones en cada nodo:

-Nodo Publisher (Publicador): El nodo publicador es una base de datos PostgreSQL normal.

Operaciones permitidas: SELECT, INSERT, UPDATE, DELETE, CREATE TABLE, ALTER TABLE y DROP TABLE

-Nodo Subscriber (Suscriptor): El nodo subscriber recibe los cambios del publisher.

Operaciones permitidas: SELECT (consultas)

Operaciones limitadas: NO debe modificar tablas replicadas (INSERT/UPDATE/DELETE)

En muchos casos, las tablas replicadas están bloqueadas o se sobrescriben por la suscripción.

2)Caída del nodo subscriber:

Parar PostgreSQL del subscriber.

En el publisher sigue funcionando normal, pero no puede enviar cambios al subscriber.

Recuperación subscriber

3)Caída del nodo publisher:

Parar el publisher.

Subscriber sigue funcionando, pero NO recibe nuevos cambio, por lo que los datos se quedan "congelados", pues no hay sincronización

Recuperación del publisher.

Durante la práctica se ha observado que:

-El publisher actúa como base de datos principal con permisos completos de lectura y escritura.

-El subscriber recibe cambios de forma automática mediante el sistema de publicaciones y suscripciones.

Cuando el subscriber cae, la replicación se interrumpe temporalmente, pero el sistema puede recuperarse sin pérdida de datos al restablecer la conexión.

Cuando el publisher cae, la replicación se detiene completamente, quedando el subscriber con datos desactualizados hasta la recuperación del nodo principal.

La replicación lógica permite una mayor flexibilidad, aunque depende de la disponibilidad del publisher para mantener la sincronización.

Cuestión 7. Conclusiones.

Durante la realización de este proyecto hemos aprendido varias cosas de crucial importancia:

- 1) Hay que desactivar el firewall de windows porque si no te vas a estar un par de horas dandote cabezazos contra la pared a intentar encontrar por que no eres capaz de conectarte si todo esta bien
- 2) Cuando creas un esclavo, metele usuarios. Si no no vas a poder entrar a arreglarlo y te tocara borrarle entero y volver a empezar (despues de otro par de horas de cabezazos a la pared porque no quiere conectarse el datasource)
- 3) Comprobar los puertos suele ser bastante importante, e incluso, mas importante aun, cuando tienes 4 configs abiertos asegurarte de cambiar las cosas en el que es. Si no ya sabemos lo que pasa
- 4) Si Chatgpt te da un consejo, 90% de las veces no es verdad, huye de ahí. Sobre todo no borres carpetas que no son porque luego te reiras cuando tengas que recuperar el config y el hba
- 5) Estar en red local facilita las cosas, si no, por lo que sea, el dolor de cabeza se vuelve exponencial con el tiempo
- 6) Esto de conectarse con ips y tal esta muy chulo, pero despues de 6 7 horas de hacer el trabajo ya no quieres volver a ver una ip en tu vida
- 7) Reiniciar el servicio de postgres puede ser la solucion a cualquier problema que sobrepasa a nuestro intelecto. No esta de mas probar.

Se deben mostrar evidencias de que ha funcionado el sistema, por eso se deben de entregar los logs y ficheros de configuración de cada equipo utilizados en la práctica.

Bibliografía

- Capítulo 18: Server Setup and Configuration.
- Capítulo 29: Server Configuration.
- Capítulo: 20.1. The pg_hba.conf File
- Capítulo 25: Backup and Restore.
- Capítulo 26: High Availability, Load Balancing, and Replication.
- Capítulo 29: Logical Replication.
- Appendix F: Additional Supplied Modules. F.38. Postgres_fdw